

C# / .NET Learning Roadmap

Topic Notes — Section 16: API Communication

Topics: 16.1 and 16.2

Date: 29 June 2026

Section 16 — API Communication

This section covers both roadmap topics: 16.1 (HttpClient and IHttpClientFactory) and 16.2 (CORS configuration).

16.1. HttpClient and IHttpClientFactory

Builds on: 7.3 (async/await -- all HTTP calls are async), 12.2 (DI -- IHttpClientFactory registered on builder.Services), 12.4 (configuration -- base URLs from config).

Why IHttpClientFactory exists

Problem 1 -- Socket exhaustion:
Each new HttpClient() holds a socket in TIME_WAIT
for ~240 seconds after Dispose().
Under load: sockets exhausted -> failures.

Problem 2 -- DNS changes ignored:
A single static HttpClient caches DNS forever.
If target IP changes, client keeps using old one.

IHttpClientFactory solves both:
-> Pools HttpResponseMessage instances
-> Rotates handlers every 2 minutes (DNS updates)
-> Reuses sockets efficiently

Named clients

```
// Program.cs -- register a named client:
builder.Services.AddHttpClient(
    "PaymentApi", client =>
    {
        client.BaseAddress =
            new Uri("https://api.payment.com");
        client.DefaultRequestHeaders
            .Add("Accept", "application/json");
        client.Timeout =
            TimeSpan.FromSeconds(30);
    });

// Use -- inject IHttpClientFactory:
public class PaymentService
{
    private readonly IHttpClientFactory _factory;

    public PaymentService(
        IHttpClientFactory factory)
        => _factory = factory;

    public async Task<PaymentResult> ChargeAsync(
        decimal amount, CancellationToken ct)
    {
        var client =
```

```

        _factory.CreateClient("PaymentApi");

var response = await client
    .PostAsJsonAsync(
        "/v1/charges",
        new { Amount = amount },
        ct);

response.EnsureSuccessStatusCode();

return await response.Content
    .ReadFromJsonAsync<PaymentResult>(
        cancellationToken: ct)!;
    }
}

```

Typed clients -- the recommended pattern

```

// Step 1 -- typed client class:
public class PaymentApiClient
{
    private readonly HttpClient _client;

    // HttpClient injected by the factory:
    public PaymentApiClient(HttpClient client)
        => _client = client;

    public async Task<ChargeResult> ChargeAsync(
        ChargeRequest request,
        CancellationToken ct)
    {
        var response = await _client
            .PostAsJsonAsync(
                "/v1/charges", request, ct);

        if (!response.IsSuccessStatusCode)
        {
            var error = await response.Content
                .ReadAsStringAsync(ct);
            throw new PaymentApiException(
                $"Charge failed "
                + $"({response.StatusCode}): "
                + error);
        }

        return await response.Content
            .ReadFromJsonAsync<ChargeResult>(
                cancellationToken: ct)
            ?? throw new PaymentApiException(
                "Empty response");
    }

    public async Task<RefundResult> RefundAsync(
        string chargeId, decimal amount,
        CancellationToken ct)

```

```

    {
        var response = await _client
            .PostAsJsonAsync(
                $"/v1/charges/{chargeId}/refund",
                new { Amount = amount }, ct);
        response.EnsureSuccessStatusCode();
        return await response.Content
            .ReadFromJsonAsync<RefundResult>(
                cancellationToken: ct)!;
    }
}

// Step 2 -- register:
builder.Services
    .AddHttpClient<PaymentApiClient>(client =>
    {
        client.BaseAddress = new Uri(
            builder.Configuration[
                "PaymentApi:BaseUrl"]!);
        client.Timeout =
            TimeSpan.FromSeconds(30);
    });

// Step 3 -- inject directly:
public class OrderService
{
    private readonly PaymentApiClient _payment;

    public OrderService(
        PaymentApiClient payment)
        => _payment = payment;
}

```

DelegatingHandler -- adding auth headers

```

public class ApiKeyDelegatingHandler
    : DelegatingHandler
{
    private readonly string _apiKey;

    public ApiKeyDelegatingHandler(
        IConfiguration config)
        => _apiKey =
            config["PaymentApi:ApiKey"]!;

    protected override async
        Task<HttpResponseMessage> SendAsync(
            HttpRequestMessage request,
            CancellationToken ct)
    {
        request.Headers
            .Add("X-API-Key", _apiKey);
        return await base
            .SendAsync(request, ct);
    }
}

```

```

}

// Register handler and attach to client:
builder.Services
    .AddTransient<ApiKeyDelegatingHandler>();

builder.Services
    .AddHttpClient<PaymentApiClient>(c =>
        c.BaseAddress = new Uri(
            "https://api.payment.com"))
    .AddHttpClientHandler<
        ApiKeyDelegatingHandler>();

```

Resilience with Polly

```

// dotnet add package
//   Microsoft.Extensions.Http.Polly

builder.Services
    .AddHttpClient<PaymentApiClient>(c =>
        c.BaseAddress = new Uri(
            "https://api.payment.com"))
    .AddTransientHttpErrorPolicy(policy =>
        policy.WaitAndRetryAsync(
            retryCount: 3,
            sleepDurationProvider: attempt =>
                TimeSpan.FromSeconds(
                    Math.Pow(2, attempt))))
    .AddTransientHttpErrorPolicy(policy =>
        policy.CircuitBreakerAsync(
            handledEventsAllowedBeforeBreaking: 5,
            durationOfBreak:
                TimeSpan.FromSeconds(30)));

// Polly retries on:
// - Network failures (connection refused)
// - 5xx server errors
// - 408 Request Timeout

```

Common request and response patterns

```

// GET -- deserialise to DTO:
var order = await _client
    .GetFromJsonAsync<OrderDto>(
        $"/api/orders/{id}", ct);

// POST with JSON body:
var response = await _client
    .PostAsJsonAsync(
        "/api/orders", createdDto, ct);
response.EnsureSuccessStatusCode();
var created = await response.Content
    .ReadFromJsonAsync<OrderDto>(
        cancellationToken: ct);

```

```
// PUT:
await _client.PutAsJsonAsync(
    $"/api/orders/{id}", updatedDto, ct);

// DELETE:
var r = await _client.DeleteAsync(
    $"/api/orders/{id}", ct);
r.EnsureSuccessStatusCode();

// Custom headers per request:
var req = new HttpRequestMessage(
    HttpMethod.Get, $"/api/orders/{id}");
req.Headers.Add("If-None-Match", etag);
var resp = await _client.SendAsync(req, ct);

// Handle errors by status code:
return response.StatusCode switch
{
    HttpStatusCode.OK =>
        await response.Content
            .ReadFromJsonAsync<OrderDto>(
                cancellationToken: ct),
    HttpStatusCode.NotFound => null,
    HttpStatusCode.Unauthorized =>
        throw new UnauthorizedException(),
    HttpStatusCode.TooManyRequests =>
        throw new RateLimitException(),
    _ => throw new ExternalApiException(
        $"Status: {response.StatusCode}")
};
```

Key Takeaways for 16.1

- Never use new HttpClient() per request -- socket exhaustion
- Never use a single static HttpClient -- ignores DNS changes
- IHttpClientFactory pools handlers, rotates every 2 min -- solves both problems
- Named clients: AddHttpClient("Name") + CreateClient("Name") -- simple
- Typed clients: AddHttpClient<TClient>() -- recommended; DI injects directly
- DelegatingHandler -- middleware for outgoing requests (auth, logging, correlation)
- Polly: AddTransientHttpErrorPolicy() for retries and circuit breakers

Pattern	Registration	Inject
Named client	AddHttpClient("Name",..)	IHttpClientFactory
Typed client	AddHttpClient<TClient>()	TClient directly
With handler	.AddHttpMessageHandler<T>	chained

Memory aid: IHttpClientFactory is a car rental agency. Never buy a car (new HttpClient). The agency gives you a pre-maintained car from the pool. Typed clients are a dedicated company car pre-configured for a specific route.

Debugging Tips

Symptom	Cause	Fix
SocketException under load	Using new HttpClient() per request	Switch to IHttpClientFactory or typed clients
Requests going to old IP after DNS change	Single static HttpClient caching DNS forever	Use IHttpClientFactory -- rotates handlers every 2 minutes
TaskCanceledException on slow APIs	Default HttpClient.Timeout (100s) mismatch	Set client.Timeout explicitly in AddHttpClient configuration
InvalidOperationException: handler already started	Modifying HttpClient headers after first request	Set headers in AddHttpClient delegate, not per-request

16.2. CORS Configuration

Builds on: 12.6 (middleware -- CORS is middleware in the pipeline), 13.1 (routing -- CORS applies before routing dispatches), 15.4 (security -- CORS introduced as part of security headers).

What is CORS and why does it exist?

CORS (Cross-Origin Resource Sharing) is a browser security mechanism. Browsers block JavaScript from making requests to a different origin. An origin = scheme + domain + port -- all three must match.

Same origin (allowed):
https://myapp.com -> https://myapp.com/api

Cross-origin (blocked by default):
https://myapp.com -> https://api.myapp.com
(different subdomain)
https://myapp.com -> http://myapp.com
(different scheme)
https://myapp.com -> https://myapp.com:8080
(different port)

IMPORTANT: CORS is browser-only.
Server-to-server calls (Postman, curl, backend)
are never blocked by CORS.

How CORS works -- preflight requests

```
// Browser sends preflight OPTIONS first:
// OPTIONS /api/orders HTTP/1.1
// Origin: https://myapp.com
// Access-Control-Request-Method: POST
// Access-Control-Request-Headers: Authorization

// Server responds (if CORS configured):
// HTTP/1.1 204 No Content
// Access-Control-Allow-Origin: https://myapp.com
// Access-Control-Allow-Methods: GET, POST
// Access-Control-Allow-Headers: Authorization
// Access-Control-Max-Age: 86400
```

```
// Browser then sends the actual request.  
// If preflight fails -> actual request blocked.
```

Defining CORS policies in Program.cs

```
builder.Services.AddCors(options =>  
{  
    // Production -- specific origins:  
    options.AddPolicy(  
        "ProductionPolicy", policy =>  
        policy  
            .WithOrigins(  
                "https://myapp.com",  
                "https://www.myapp.com",  
                "https://admin.myapp.com")  
            .WithMethods(  
                "GET", "POST", "PUT",  
                "DELETE", "PATCH")  
            .WithHeaders(  
                "Content-Type",  
                "Authorization",  
                "X-Correlation-Id")  
            .AllowCredentials()  
            .SetPreflightMaxAge(  
                TimeSpan.FromHours(24)));  
  
    // Development -- allow any origin:  
    options.AddPolicy(  
        "DevelopmentPolicy", policy =>  
        policy  
            .AllowAnyOrigin()  
            .AllowAnyMethod()  
            .AllowAnyHeader());  
    // AllowAnyOrigin + AllowCredentials  
    // is NOT allowed -- browser rejects it  
  
    // Partner integration:  
    options.AddPolicy(  
        "PartnerApiPolicy", policy =>  
        policy  
            .WithOrigins(  
                "https://partner-a.com",  
                "https://partner-b.com")  
            .WithMethods("GET", "POST")  
            .WithHeaders(  
                "Content-Type",  
                "X-Partner-Key")  
            .WithExposedHeaders(  
                "X-Rate-Limit-Remaining",  
                "X-Total-Count"));  
});
```

Applying CORS in the middleware pipeline

```
// Environment-based selection:
```



```

if (app.Environment.IsDevelopment())
    app.UseCors("DevelopmentPolicy");
else
    app.UseCors("ProductionPolicy");

// CRITICAL -- correct order:
app.UseRouting();           // 1. match routes
app.UseCors();              // 2. CORS headers
app.UseAuthentication();    // 3. authenticate
app.UseAuthorization();     // 4. authorise
app.MapControllers();       // 5. dispatch

// UseCors MUST be:
// AFTER UseRouting()
// BEFORE UseAuthentication()

```

Per-endpoint CORS

```

// Different policy on specific controller:
[EnableCors("PartnerApiPolicy")]
[ApiController]
[Route("api/partner")]
public class PartnerController : ControllerBase
{
    [HttpGet("orders")]
    public IActionResult GetOrders() { }

    // Disable CORS on specific action:
    [DisableCors]
    [HttpGet("internal")]
    public IActionResult InternalOnly() { }
}

// Minimal API:
app.MapGet("/api/orders", GetOrders)
    .RequireCors("ProductionPolicy");

```

Dynamic origins from configuration

```

// appsettings.json:
// { "Cors": { "AllowedOrigins": [
//     "https://myapp.com",
//     "https://www.myapp.com" ] } }

var allowedOrigins = builder.Configuration
    .GetSection("Cors:AllowedOrigins")
    .Get<string[]>() ?? [];

builder.Services.AddCors(options =>
    options.AddPolicy(
        "ConfiguredPolicy", policy =>
        {
            if (builder.Environment.IsDevelopment())
                policy
                    .AllowAnyOrigin()

```

```

        .AllowAnyMethod()
        .AllowAnyHeader();
    else
        policy
            .WithOrigins(allowedOrigins)
            .AllowAnyMethod()
            .AllowAnyHeader()
            .AllowCredentials();
    }));

```

Common CORS mistakes

AllowAnyOrigin + AllowCredentials:
 policy.AllowAnyOrigin().AllowCredentials()
 Browser rejects this combination.
 Fix: use WithOrigins(specific) with credentials

UseCors after MapControllers:
 app.MapControllers();
 app.UseCors("Policy"); // too late!
 Fix: UseCors before MapControllers

AllowAnyOrigin in production:
 Any website can call your API.
 Fix: whitelist specific origins only

Missing OPTIONS handling:
 Auth middleware rejects preflight before CORS.
 Fix: UseCors() before UseAuthentication()

Not exposing needed headers:
 Browser blocks custom response headers.
 Fix: .WithExposedHeaders("X-Total-Count")

Testing CORS configuration

```

[Fact]
public async Task
    GetOrders_AllowedOrigin_ReturnsCorsHeader()
{
    // Simulate browser preflight:
    var request = new HttpRequestMessage(
        HttpMethod.Options, "/api/orders");
    request.Headers.Add(
        "Origin", "https://myapp.com");
    request.Headers.Add(
        "Access-Control-Request-Method", "GET");

    var response = await _client
        .SendAsync(request);

    Assert.True(response.Headers.Contains(
        "Access-Control-Allow-Origin"));
    Assert.Equal(
        "https://myapp.com",

```

```

        response.Headers
            .GetValues(
                "Access-Control-Allow-Origin")
            .First());
    }

[Fact]
public async Task
    GetOrders_BlockedOrigin_NoCorsHeader()
{
    var request = new HttpRequestMessage(
        HttpMethod.Options, "/api/orders");
    request.Headers.Add(
        "Origin", "https://attacker.com");
    request.Headers.Add(
        "Access-Control-Request-Method", "GET");

    var response = await _client
        .SendAsync(request);

    // No header for blocked origin:
    Assert.False(response.Headers.Contains(
        "Access-Control-Allow-Origin"));
}

```

Key Takeaways for 16.2

- CORS is a browser-only mechanism -- server-to-server calls are never blocked
- Origin = scheme + domain + port -- all three must match for same-origin
- Browser sends OPTIONS preflight for non-simple requests -- API must handle it
- Define named policies with AddCors(options => options.AddPolicy(...))
- UseCors() must be AFTER UseRouting() and BEFORE UseAuthentication()
- AllowAnyOrigin() cannot be combined with AllowCredentials()
- WithExposedHeaders() to expose custom response headers to browser JS
- Read allowed origins from config -- never hardcode in production

Method	Effect
WithOrigins("https://...")	Allow specific origins
AllowAnyOrigin()	All origins (dev only)
WithMethods("GET", "POST")	Allow specific methods
AllowAnyMethod()	Allow all methods
WithHeaders("Auth")	Allow specific headers
AllowAnyHeader()	Allow all headers
AllowCredentials()	Allow cookies + auth
WithExposedHeaders("X-T")	Expose response headers
SetPreflightMaxAge(span)	Cache preflight duration

Memory aid: CORS is a club guest list policy. The club (your API) posts the policy at the door (CORS headers). The bouncer (browser) checks if the guest origin is on the list before letting the response through. The bouncer only exists at the browser entrance -- backend-to-backend calls bypass the bouncer entirely. The preflight is the guest calling ahead to check the dress code.

Debugging Tips

Symptom	Cause	Fix
CORS error in browser despite policy configured	UseCors() registered after MapControllers()	Move UseCors() between UseRouting() and MapControllers()
Preflight returns 401	UseAuthentication() runs before UseCors()	Move UseCors() before UseAuthentication()
AllowCredentials throws InvalidOperationException	AllowAnyOrigin() combined with AllowCredentials()	Use WithOrigins(specific URLs) when using credentials
Custom response header not visible to browser JS	Header not in WithExposedHeaders()	Add .WithExposedHeaders("Your-Header-Name") to the policy
CORS works in dev but not production	Different origins or environment mismatch	Check ASPNETCORE_ENVIRONMENT; verify origin list matches prod URLs

Section 16 -- Complete Key Takeaways

Topic	Core concept	Key API
-----+-----+-----		
16.1 HttpClient	Factory pools handlers	IHttpClientFactory
	Named + typed clients	AddHttpClient<T>
	Polly resilience	WaitAndRetryAsync
16.2 CORS	Browser-only mechanism	AddCors, UseCors
	Preflight OPTIONS	WithOrigins
	Policy per endpoint	[EnableCors]

Section 16 complete. Both roadmap topics (16.1 and 16.2) covered. Say next to begin Section 17: Clean Architecture / Project Structuring (17.1 Layered architecture, 17.2 Separation of concerns, 17.3 Repository and Unit of Work in practice).